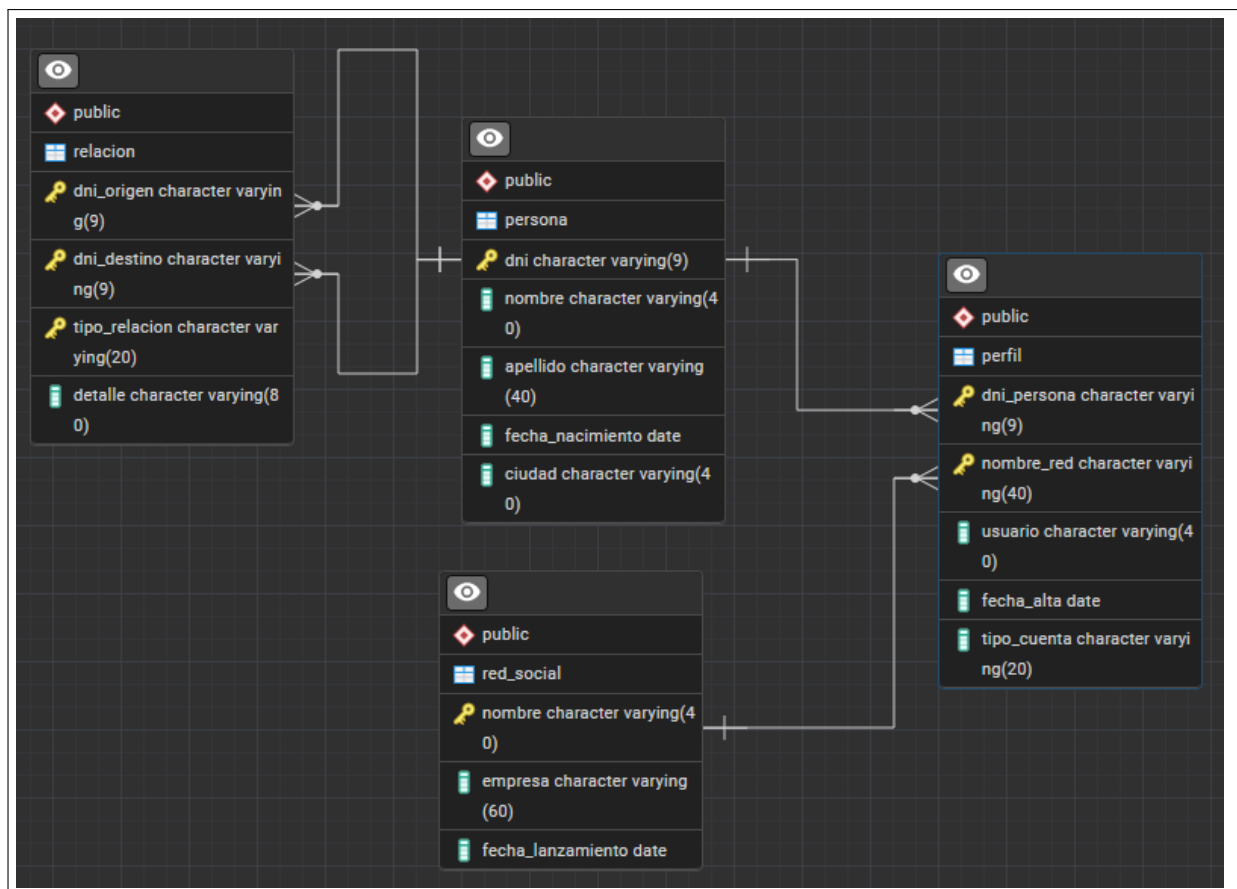


Segundo parcial - Mayo 2026

Base de datos

El siguiente diagrama relacional muestra una base de datos sencilla para registrar personas, perfiles en redes sociales y relaciones entre personas.



Puedes descargar los ficheros `redes_sociales_ddl.sql` y `redes_sociales_dml.sql`, que contienen los scripts necesarios para crear y rellenar la base de datos.

Modelo relacional

La base de datos contiene las siguientes tablas:

- `persona(dni, nombre, apellido, fecha_nacimiento, ciudad)`
- `red_social(nombre, empresa, fecha_lanzamiento)`
- `perfil(dni_persona, nombre_red, usuario, fecha_alta, tipo_cuenta)`
- `relacion(dni_origen, dni_destino, tipo_relacion, detalle)`

La tabla `perfil` representa los identificadores o cuentas que una persona tiene en distintas redes sociales. La tabla `relacion` representa relaciones entre personas. El atributo `tipo_relacion` puede

tomar valores como amistad, familiar, entrenamiento o profesional. El atributo detalle recoge información propia de la relación: por ejemplo, el grado de parentesco si es familiar o la actividad si es de entrenamiento.

Instrucciones

Contesta las siguientes preguntas. Para ello **escribe un fichero que contenga el script SQL con tus respuestas, indicando claramente qué respuesta corresponde a cada ejercicio y justificando brevemente tus decisiones. Añade toda esa información como comentarios en tu script, de manera que se pueda ejecutar directamente.**

OBS.- recuerda que se puede utilizar `void` como indicador de que una función no devuelve nada

1. Escribir una función de nombre `perfiles_en_red(red text)` que devuelva la consulta formada por las personas que tienen perfil en una red social cuyo nombre se proporciona como argumento. La consulta debe devolver el DNI, nombre, apellido, usuario, tipo de cuenta y nombre de la red social. Si no hay resultados, debe mostrar un mensaje informando de esa circunstancia. **(1,5 puntos)**

Añade además dos sentencias que invoquen la función: una que muestre resultados y otra que muestre el mensaje correspondiente a que no hay resultados. **(0,5 puntos)**

2. Escribe una función `listar_relaciones_persona(persona_dni text)` que tome como argumento el DNI de una persona y recupere la información de sus relaciones personales.

Para ello debes: **(1,5 puntos)**

- Diseñar una consulta que devuelva el nombre y apellido de la persona origen, el nombre y apellido de la persona relacionada, el tipo de relación y el detalle de la relacion.
- Recorrer las filas devueltas por esa consulta con un bucle específico que no use cursores explícitos.
- Para cada fila devuelta por esta consulta, mostrar un mensaje con esa información.

Añade dos sentencias SQL que prueben esa función: una para la que haya datos como resultado de la consulta y otra para la que no. **(0,5 puntos)**

3. Escribe una función `mostrar_redes_con_num_personas()` que recorra con un cursor explícito todas las redes sociales. Para cada red debe mostrar toda la información almacenada sobre la red y añadir el número de personas que tienen cuenta en ella. **(1,5 puntos)**

Añade una sentencia SQL que pruebe el funcionamiento de esta función. **(0,5 puntos)**

4. A continuación vas a gestionar un disparador sobre la tabla `relacion`, de tal manera que se guarde constancia de cada operación de inserción, actualización o borrado que se realice en la tabla.

- a. Crear una tabla de log de operaciones sobre relaciones con información del momento en el que se hace la operación, el tipo de operación realizada, y todos los datos de la fila insertada, modificada o borrada en la tabla `relacion`. **(1 punto)**
- b. Escribir una función que sirva como disparador sobre la tabla `relacion`, y que inserte en la tabla de log la información correspondiente a la operación realizada. **(2 puntos)**
- c. Proporcionar el código SQL necesario para vincular esa función con las operaciones sobre la tabla `relacion`. **(0,5 puntos)**
- d. Aportar 2 instrucciones de inserción, 2 de actualización y 2 de borrado que permitan probar la funcionalidad del disparador. **(0,5 puntos)**